Workshop - Spec Coding with Apache Cassandra®

Aaron Ploetz
Engineer

aaron.ploetz@ibm.com

Agenda

Timeline

Introduction	Cassandra, Astra DB, and Spec Coding!
Use Case	What problem are we solving?
Data Model	Designing tables, analyzing prospective queries.
Data Loading	Putting data into the DB
Workflow	Use your AI coding agent tool to build services that access the data.
Closing	Finishing remarks and resources.

Command Prompt

File Edit View Terminal Tabs Help

```
$ whoami
```

Aaron Ploetz
developer advocate
DataStax / IBM

committer:
Apache Cassandra® project

engineer:
Grainger, AccuLynx, Target

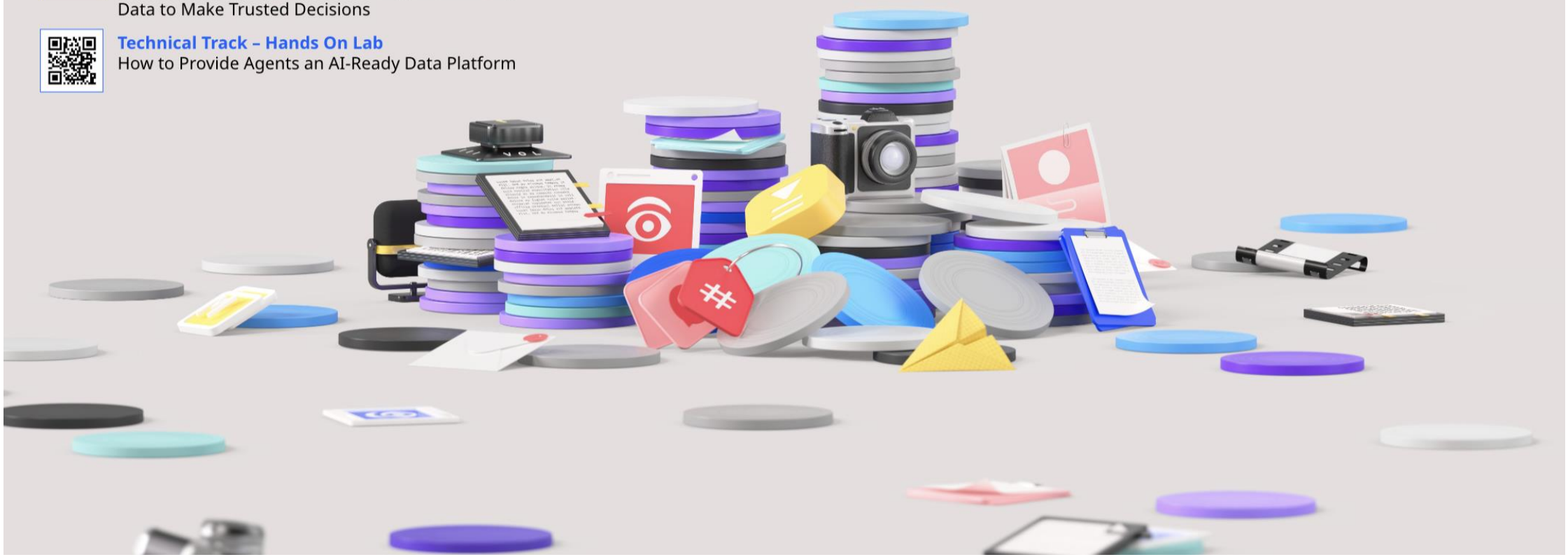
author:
- Seven NoSQL Databases in a Week
- Mastering Apache Cassandra 3.x
- Code with Java 21
- Think Like a Programmer

Join Us to Build Trusted, Context-Aware AI

Two Tracks. One AI-Ready Data Foundation.
June 10 | New York, NY

Leadership Track – Executive Strategy Workshop
The Context Gap: How to Provide Agents AI-Ready Data to Make Trusted Decisions

Technical Track – Hands On Lab
How to Provide Agents an AI-Ready Data Platform



Astronaut & Spaceflight Tracker


```
{
  "name": "Jim Lovell",
  "dob": "1928-03-25",
  "birthplace": "Cleveland, OH",
  "university_name": "US Naval Academy"
}
```

```
{
 "items": [
  {
   "mission_name": "Apollo 11",
   "astronaut_name": "Buzz Aldrin",
   "ship_name": "Columbia",
   "mission_start_date": "1969-07-16T18:32:00Z",
   "mission_end_date": "1969-07-24T21:50:35Z"
  },
  {
   "mission_name": "Apollo 11",
   "astronaut_name": "Michael Collins",
   "ship_name": "Columbia",
   "mission_start_date": "1969-07-16T18:32:00Z",
   "mission_end_date": "1969-07-24T21:50:35Z"
  },
  {
   "mission_name": "Apollo 11",
   "astronaut_name": "Neil Armstrong",
   "ship_name": "Columbia",
   "mission_start_date": "1969-07-16T18:32:00Z",
   "mission_end_date": "1969-07-24T21:50:35Z"
  }
 ],
 "count": 3
}
```

Spec first, code second.

Write detailed specifications — requirements, data models, API contracts — and use them as executable blueprints for an AI coding agent. The spec is your source of truth. The agent generates the code.

THE KEY INSIGHT

Planning matters even more with an agent. Iterate on the plan first, then hand it off. The spec is the first artifact you and the AI build together.

NOT VIBE CODING

No “just make it work” prompting. You define what right looks like **before** any code is written.

NOT WATERFALL

Each step is iterative. Refine requirements, sharpen the contract, adjust tasks — as many loops as you need.

THIS WORKSHOP

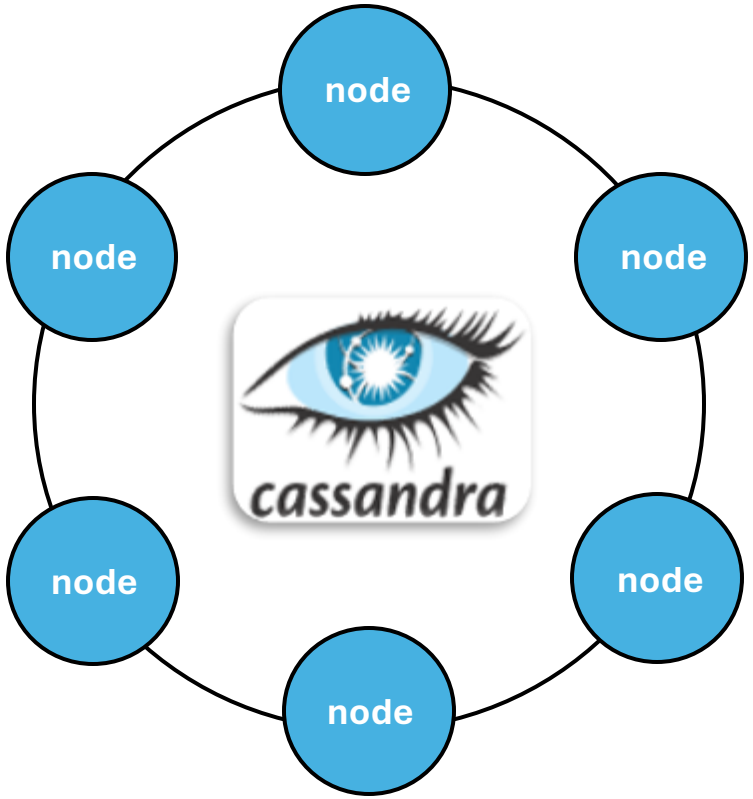
Three steps: requirements, spec (design + OpenAPI), then tasks + code. You’ll build a real app using this workflow.

“Spec-driven development is one of the most important practices to emerge in 2025.” — Thoughtworks Technology Radar

What is Cassandra?

Apache Cassandra®

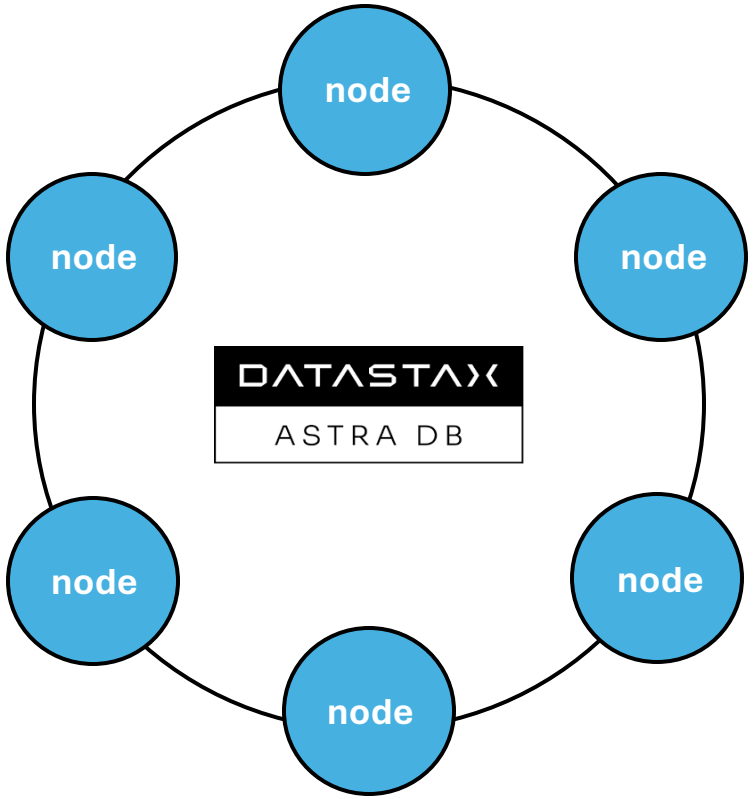
- Highly Available NoSQL database (AP).
- Geographic distribution.
- Scalable read and write performance.
- Designed to run on commodity hardware.
- All nodes are peers.



What is Astra DB?

Astra DB

- Cloud native, managed service built on **Apache Cassandra®**.
- No operational overhead.
- Runs in all 3 major clouds.
- “No touch” Scalability.



A working app, a reusable workflow, and a repo full of proof.

A working app

Service layer backed by Astra Data API, tested and demoed.

A repeatable workflow

Requirements, design, OpenAPI contract, tasks. Spec first, code second.

Evidence of progress

Tests pass, demo script runs, CI is green. Not just code — proof it works.

Astronaut & Spaceflight Tracker

- Stores data on individual astronauts.
- Stores data on spaceflight missions.
- Allows CRUD operations on both.
- Serves restful endpoints to each underlying operation.

```
git clone https://github.com/aar0np/spec_coding_astronaut_app.git
```

Astronaut & Spaceflight Tracker Pre-requisites

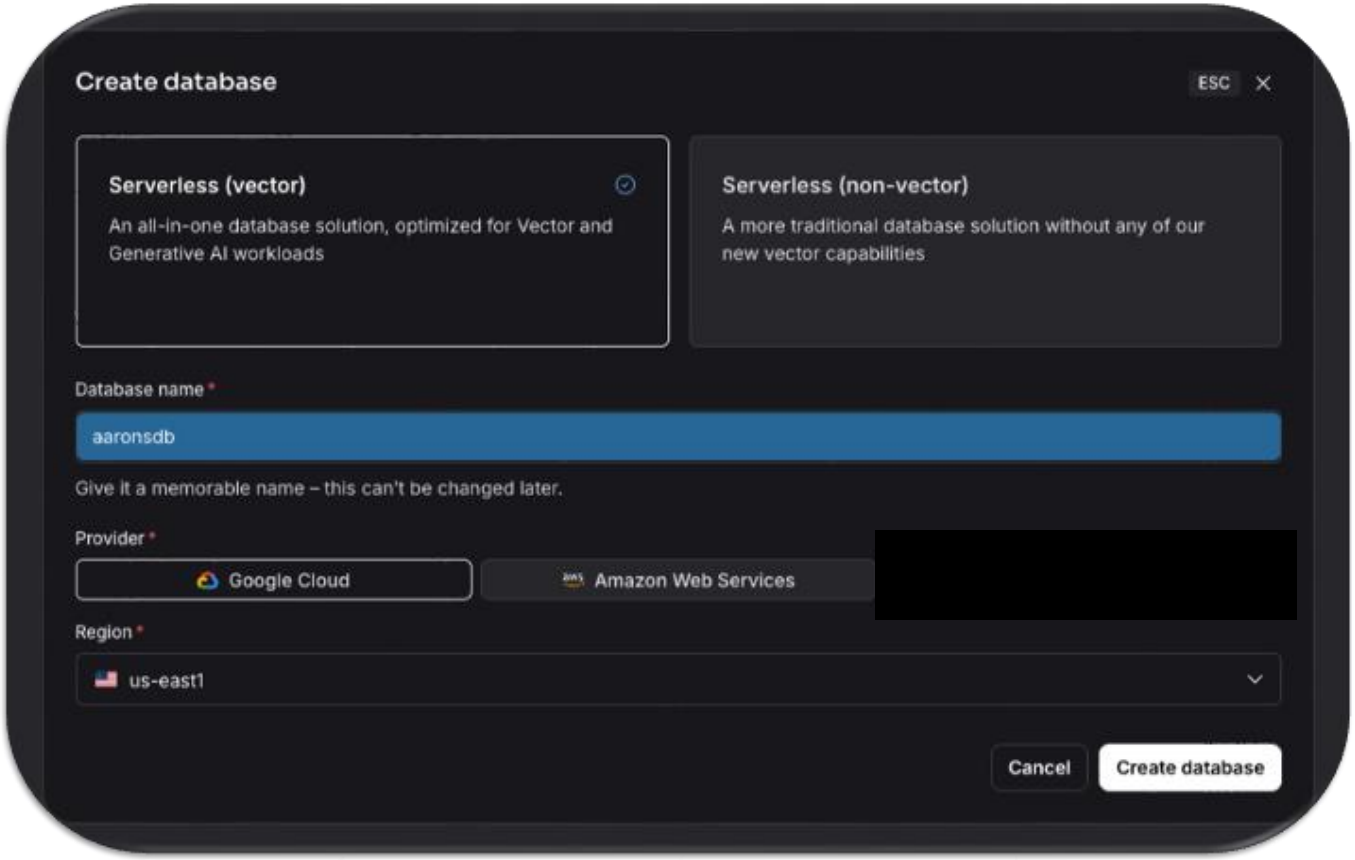
- Python 3.10 or 3.11
- An Astra DB account
- An agentic coding tool (Cursor, Claude Code, IBM Bob)

Exercise #1
Create an Astra DB Database

Create an Astra DB Database

Click the "Create database" button (upper right corner) to create a new Astra DB instance:

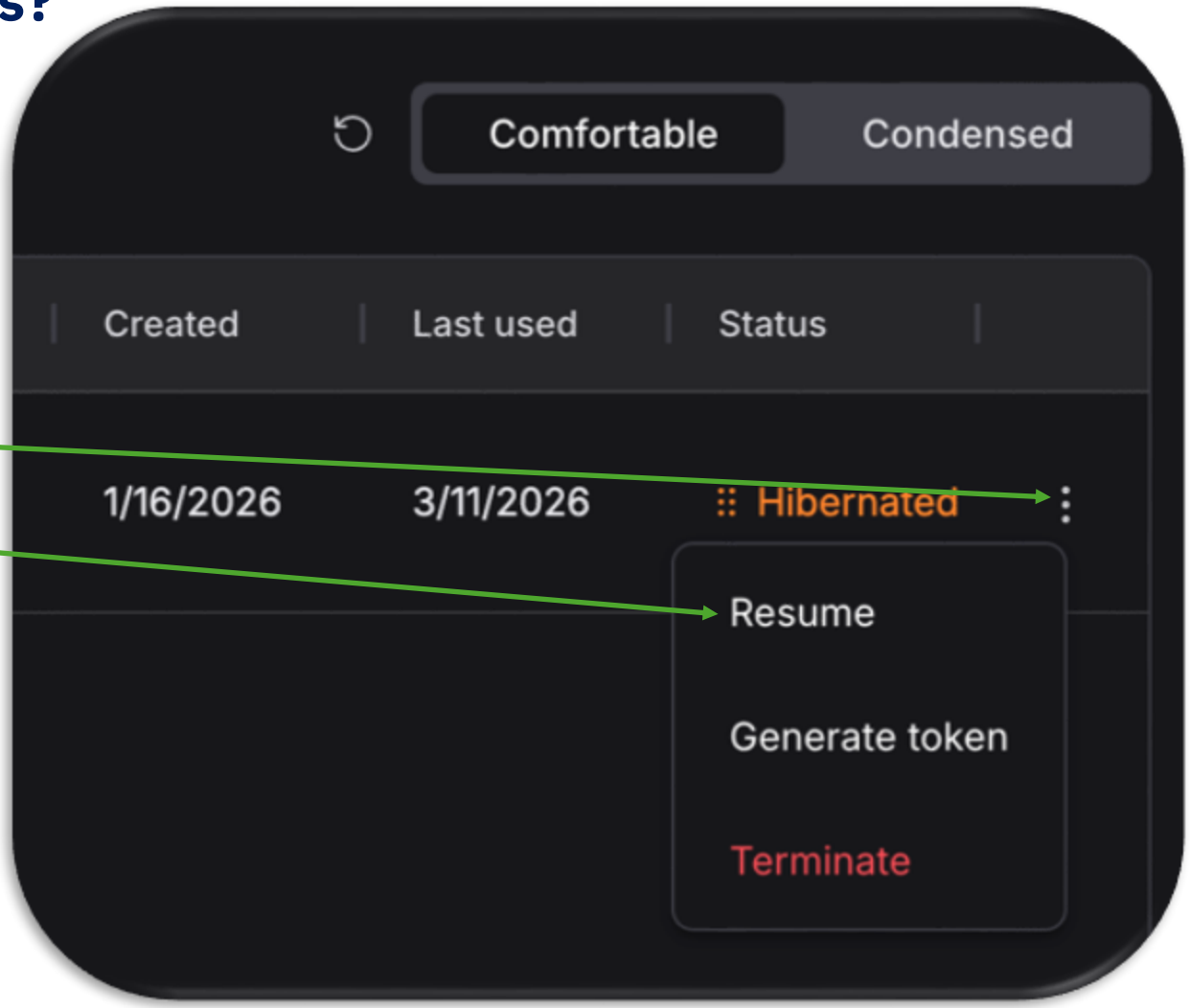
- Select "Serverless (Vector)"
- name: *whatever*
- Pick a cloud provider.
- Select a "free" region.



Is your database in a "Hibernated" status?

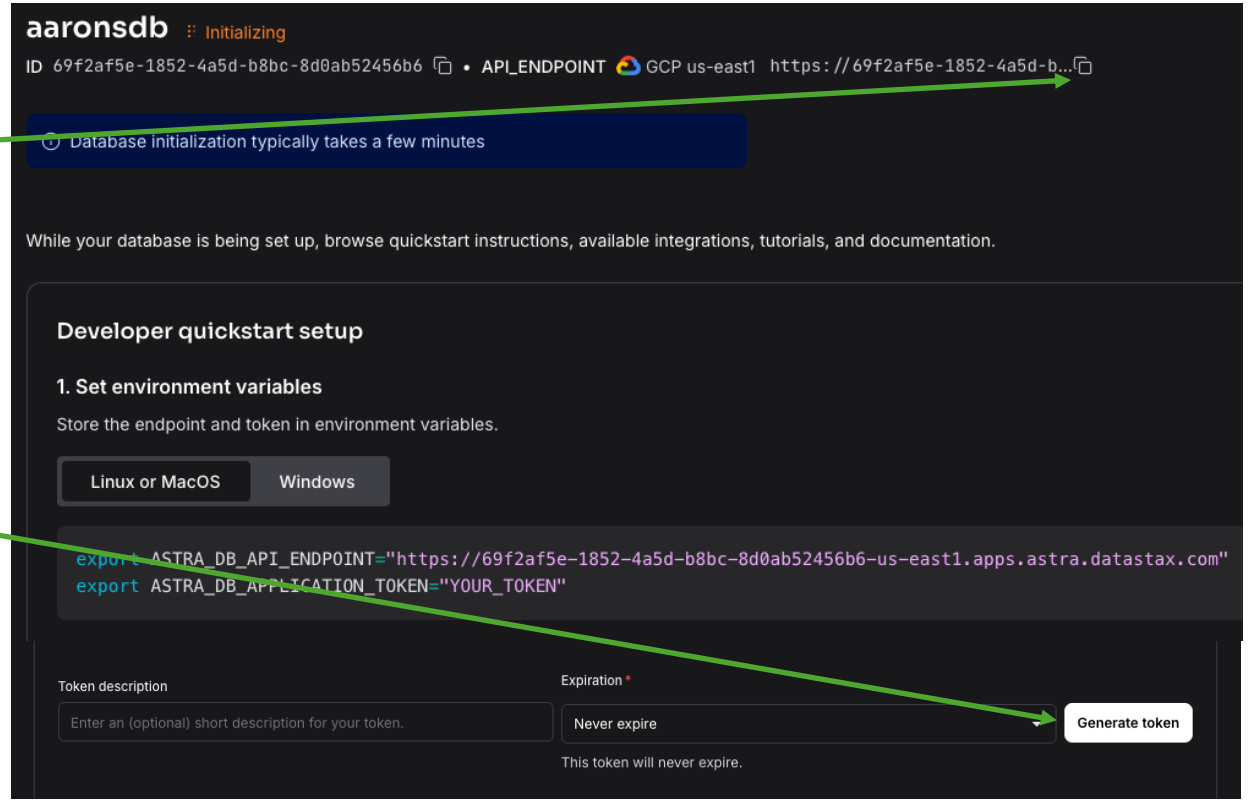
To resume an Astra DB Database

- Click the "3 dots" menu
- Select "resume"



Create your credentials

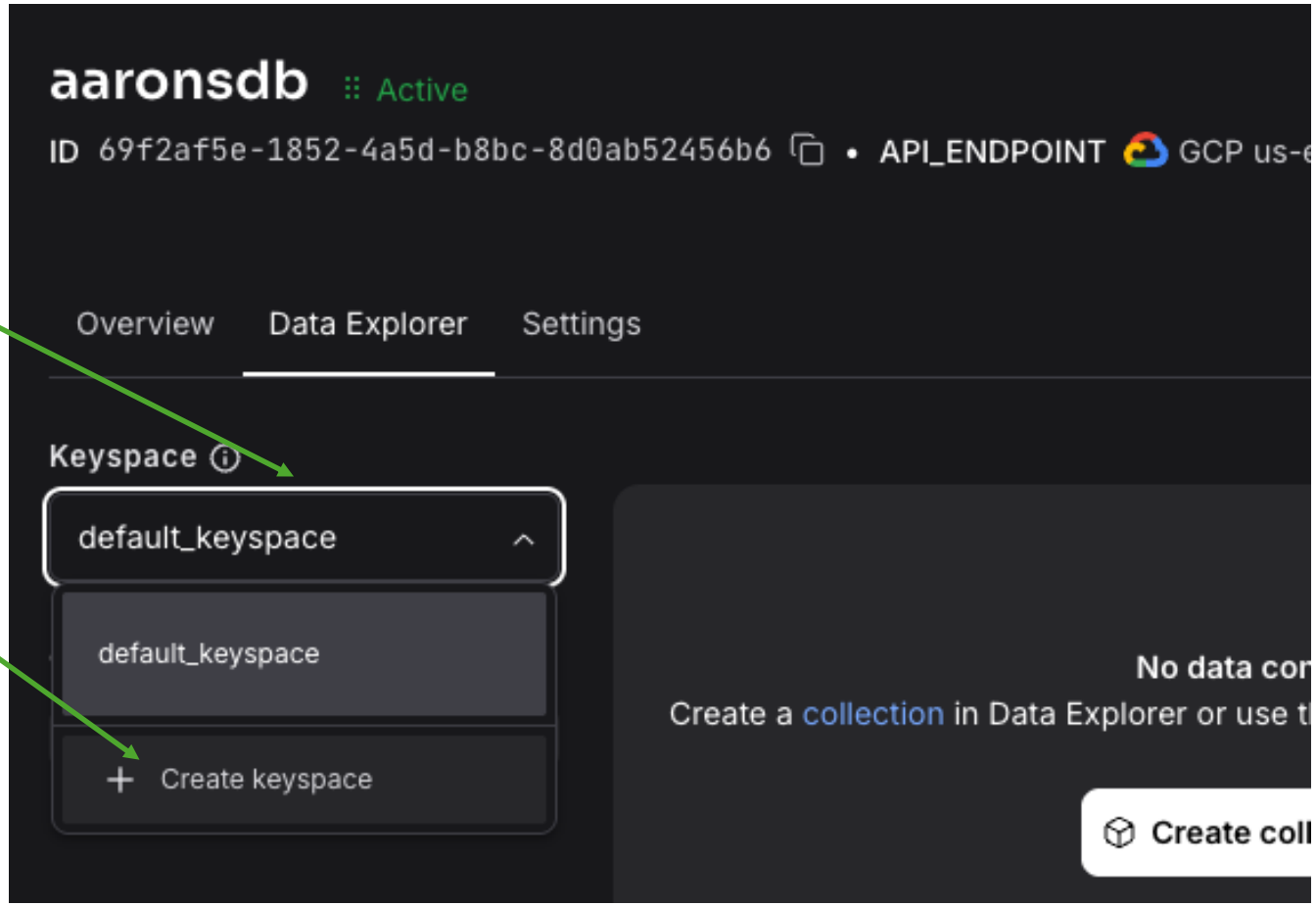
- Once available, copy the API_ENDPOINT by clicking its copy button.
- Click the "Generate token" button (lower right corner) to create a new access token.



Be sure to save the endpoint and token for later!

Create a keyspace

- Click the "Keyspace" menu.
- Select "Create keyspace" and name it "astronaut_app"



DO NOT click "Create collection!"

Exercise #2
Create schema

CQL console

Astronaut & Spaceflight Tracker Data Model

```
CREATE TABLE astronauts (
 name TEXT PRIMARY KEY,
 dob DATE,
 birthplace TEXT,
 university_name TEXT);
```

```
CREATE TABLE astronauts_by_mission (
 astronaut_name TEXT,
 ship_name TEXT,
 mission_name TEXT,
 mission_start_date TIMESTAMP,
 mission_end_date TIMESTAMP,
 PRIMARY KEY (mission_name, astronaut_name);
```

DATASTAX

ASTRA DB

Exercise #3
Create environment file

Install dependencies

```
pip install astrapy
pip install poetry
```

Environment file

- Copy environment example file to `.env`.

```
cp .env.example .env
```

- Edit the `.env` file, adding your credentials.

```
ASTRA_DB_API_ENDPOINT=https://...
ASTRA_DB_APPLICATION_TOKEN=AstraCS:...
ASTRA_DB_KEYSPACE=astronaut_app
```

Exercise #4
Load data

Astronaut & Spaceflight Tracker

- Data loaders in the `loaders/` directory.
- CSV files in the `data/` directory.
- Both loaders import the `app/core/config.py` module.
- Be sure to set the credential variables in the `.env` file!

```
python3 loaders/load_astronauts.py
```

```
python3 loaders/load_astronauts_by_mission.py
```

Check the database

CQL console

CQL commands to try:

```
use astronaut_app;
```

```
desc table astronauts_by_mission;
```

```
SELECT * FROM astronauts_by_mission LIMIT 5;
```

Building the Service Layer

Three steps. Iterate each one until it's right.

1	**Requirements**	**Output** `requirements.md`	↻ until specific, testable, complete
2	**Spec**	**Output** `design.md` `openapi.yaml`	↻ until spec validates + covers reqs
3	**Tasks + Implement**	**Output** `todo.md` `src/` `tests/`	↻ until all tests pass

What "iterate" looks like

Prompt the agent. Review the output. Refine the prompt. Run again.

Don't move to the next step until you're happy with the current output.

The spec files are your ground truth — they survive every iteration.

Goal + Context + Constraints + Examples = Awesome Prompt

Goal

What you want built/changed (definition of done). **Bonus:** Defined Role

Context

Stack + relevant code/files

Constraints

Must/never rules (deps, API, perf, style)

Examples

Inputs → outputs + edge cases

Pro Tip: Have the agent interview you. “You are a product manager. Interview me about X”

Context is everything

Better context = fewer wrong assumptions, less refactoring, faster correct output

Include only what matters:

- The relevant files / interfaces
- Docs
- Websites

MCP

Avoid if possible

Contributes to context bloat

Context7 MCP maybe an exception

Skills

Small bundles of docs + instructions

Can dynamically load MCPs

Easy to make

https://docs.datastax.com/en/astra-cli/index.html

https://docs.datastax.com/en/astra-db-serverless/api-reference/dataapiclient.html

Pro Tip: Have the agent create a doc directory and scrape docs to create local docs on everything

Step 1 Requirements

↻ iterate until done

WHAT WE'RE DOING

Write a clear set of requirements for your project. Think user stories, acceptance criteria, and constraints. Give each requirement a testable ID (e.g. REQ-001). Avoid anything technical!

HOW TO PROMPT

```
# Tell the agent what you're building

I'm building a [your project]. Write requirements in requirements.md with:

- Numbered IDs (REQ-001, REQ-002 ...)

- Acceptance criteria for each

- Personas and User flows

- Do not include any implementation details such as code or technology choices
```

DONE =

`requirements.md` exists with numbered, testable requirements that you've reviewed and are happy with.

Pro Tip: Check for scope creep. That happens. A LOT

Finished early? Ask the agent to add edge cases or "what could go wrong" scenarios.

Exercise #5a
Building the requirements.md file

What you should have now

Your repo

- ✅ `requirements.md`
- ✅ `.env`
- ✅ `.gitignore`

SAMPLE REFINEMENT PROMPT

```
Review requirements.md.  Are there
any gaps, ambiguities, or missing
edge cases?  Fix them and update
the file.
```

Next up → Step 2: Spec (design.md + openapi.yaml)

Step 2

Spec (Design + OpenAPI)

↻ iterate until done

WHAT WE'RE DOING

Turn your requirements into a data model (design.md) and an OpenAPI contract (openapi.yaml). The OpenAPI file becomes the single source of truth for your API.

HOW TO PROMPT

```
# Feed it the requirements as context

Read requirements.md.  From those requirements:

1. Create design.md with data model + collections, web frontend, and backend service layer which uses Astra DB.

2. Create openapi.yaml (OpenAPI 3.1) with

   all endpoints, schemas, and error responses

3. Map each endpoint back to a REQ-ID
```

DONE =

`design.md` + `openapi.yaml` that validates and maps every requirement to an endpoint.

Finished early? Ask the agent to validate the OpenAPI spec and add example responses for each endpoint.

Exercise #5b

Building the design.md and OpenAPI.yaml files

What you should have now

Your repo

- ✅ `requirements.md`
- ✅ **`design.md`** (new)
- ✅ **`openapi.yaml`** (new)
- ✅ `.env`
- ✅ `.gitignore`

SAMPLE REFINEMENT PROMPT

```
Validate openapi.yaml.  Check that
every REQ-ID in requirements.md is
covered by at least one endpoint.
List any gaps and fix them.
```

Next up → Step 3: Tasks + Implement (todo.md + working code)

Step 3 Tasks + Implement

↻ iterate until done

WHAT WE'RE DOING

Break the spec into a task list (todo.md), then have the agent implement each task: server, frontend, Astra DB integration, and tests. Run tests after each task.

HOW TO PROMPT

```
# Give it all the context it needs

Read requirements.md, design.md, openapi.yaml.

1. Create todo.md breaking the spec into tasks

2. Implement each task.  Use .env for creds.

3. Write tests and run them after each task

4. Mark each task done in todo.md when it passes
```

DONE =

A working server + frontend connected to Astra DB. All tests passing. Every task checked off in `todo.md`.

Pro Tip: Use github issues instead of a file. Using the gh command line tool is extra bonus tip

Finished early? Ask the agent to add error handling, improve the UI, or write integration tests against Astra.

Exercise #5c
Building the todo.md file

What you should have now

Your repo

- ✅ `requirements.md`
- ✅ `design.md`
- ✅ `openapi.yaml`
- ✅ `todo.md` (new)
- ✅ `src/` `tests/` (new)

SAMPLE REFINEMENT PROMPT

```
Run all tests.  For any that fail:
1. Read the error output
2. Check openapi.yaml for the expected
   behavior
3. Fix the implementation, not the test
4. Re-run until green
```

You did it! A working app, a reusable workflow, and a repo full of proof — built with spec-driven development.

Exercise #5d
Execute each TODO task

Try testing some of the endpoints

```
curl –X GET-s "http://localhost:8000/astronauts/Pete%20Conrad" | jq
{
  "name": "Pete Conrad",
  "dob": "1930-05-02",
  "birthplace": "Philadelphia, PA",
  "university_name": "Princeton University"
}
```

```
curl  -X POST -s "http://localhost:8000/astronauts" \
  -H "Content-Type: application/json" \
  -d '{"name": "Victor Glover", "dob": "1976-04-30",
"birthplace": "Pomana, CA", "university_name":
"California Polytechnic State University"}'
```

Your specific endpoints and payloads may be different!

What else could you add?

Endpoints for...

- Equipment tracking
- Payload management
- Mission planning

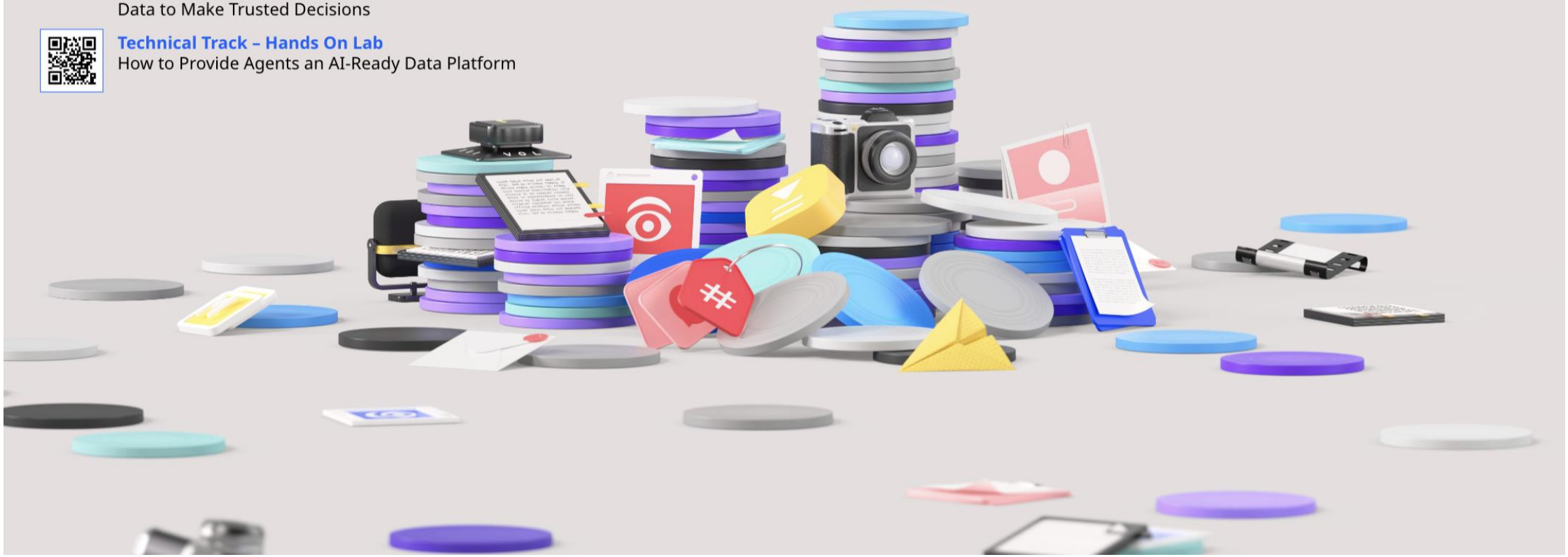
Or build a web front end to serve the data?

Join Us to Build Trusted, Context-Aware AI

Two Tracks. One AI-Ready Data Foundation.
June 10 | New York, NY

Leadership Track – Executive Strategy Workshop
The Context Gap: How to Provide Agents AI-Ready Data to Make Trusted Decisions

Technical Track – Hands On Lab
How to Provide Agents an AI-Ready Data Platform



Thank you!